

Task 1 - Design The Algorithm

Problem Definition: The university maintains student records identified by a unique Registration Number. The system must determine whether a requested reg no exists. If it exist, retrieve the corresponding student record.

The Database may grow from:

1,000 $\rightarrow$ 10,000 $\rightarrow$ 100,000 $\rightarrow$ 1,000,000+ records.

Approach 1 :- Linear Search

Design strategy: sequential search

Basic idea:

- *) The required register number is found.
- *) The entire database has been checked

Example: The requested REG NO: 520.

Algorithm checks:

REG NO 1 $\rightarrow$ NO

REG NO 5 $\rightarrow$ NO

REG NO 520 $\rightarrow$ Found

Pseudo code:

ALGORITHM Linear_search (records, target)

FOR each record in records DO

IF record.registrationNumber = target THEN

RETURN record

END FOR

RETURN "Record not found"

Preprocessing:

*) None, The database can remain in its original order

Complexity:

Case	Time complexity
Best case	$O(1)$
Avg case	$O(n)$
Worst case	$O(n)$
Space complexity	$O(1)$

∴ Therefore, although simple it may become inefficient for university

APPROACH 2 - RECURSIVE BINARY SEARCH.

Design strategy: Divide and conquer

Basic idea:

*) Binary search works on a sorted list of registration numbers

Preprocessing:

*) The record must first be sorted by registration number.

PSEUDO CODE:

$mid \leftarrow \text{Floor}((low + high) / 2)$

IF records[mid]. regno = target THEN

RETURN records[mid]

complexity:

operation	avg case	worst case
Hash-table construction	$O(n)$	$O(n)$
Search	$O(1)$	$O(n)$
insertion	$O(1)$	$O(n)$

∴ Hashing is particularly suitable for university's workload

RECOMMENDED DESIGN:

*) HASH TABLE

Because, it demonstrates that the design considers Database size, Searching frequency, preprocessing, scalability, Reliability.

TASK 2 - IDENTIFY THE BASIC OPERATION

Algorithm	Basic operation	dominates.
Linear search	Req-no comparison	It may be performed once for ever sec
Recursive Binary search	Req-no comparison at middle element	Each comparison eliminates approximately
Hash table Search	Hash calculation + key comparison	It determines the location of required record.

ELSE IF target < records[mid]. registrationNumber THEN
 records,
 target,
 low,
 mid - 1)

END ALGORITHM

Complexity: For a single search after sorting

Case	Time complexity
Best	$O(1)$
Avg	$O(\log n)$
worst	$O(\log n)$
Recursive	$O(\log n)$

Preprocessing:

Sorting $\rightarrow O(n \log n)$

Binary search $\rightarrow O(\log n)$ per request

$\therefore$ maintaining sorted data can reduce additional overhead.

APPROACH 3: HASH TABLE SEARCH.

Design strategy: Hashing

Preprocessing: The original student database must be transformed into a hash-indexed structure.

Pseudo code:

ALGORITHM Hashsearch (HashTable, target)

IF HashTable [index] contains target THEN
 RETURN corresponding student record

END ALGORITHM.

2-1 Linear Search

Basic operation: records[i].registrationNumber == target

Occurrence:.

For each record

IF record.registrationNumber == target

RETURN record

appropriate operation:.

IF there are n records, the comparison can occur

1 time $\rightarrow$ best case

$n/2$ times $\rightarrow$ approximately avg case

n times $\rightarrow$ worst case

Behaviour:.

IF the database grows from:

1,000 $\rightarrow$ 10,000 $\rightarrow$ 100,000 $\rightarrow$ 1,000,000

the max number of comparisons grown proportionally.

1,000 $\rightarrow$ 10,000 $\rightarrow$ 100,000 $\rightarrow$ 1,000,000.

2-2 RECURSIVE BINARY SEARCH

Basic operation: target > records[mid].registrationNumber

occurrence:.

mid = (low + high) / 2

IF records[mid] == target

return record

ELSE

search right half.

appropriate operation:.

$$\frac{n}{2^k} = 1$$

$$n = 2^k \quad (\text{Taking logarithm})$$

Then

$$k = \log_2 n.$$

2.3 Hash Table Search

Basic operation: compute the hash value of registration number and using it to access the corresponding hash-table location, followed by key comparison

occurrence:.

Index $\leftarrow$ Hash(target)

IF HashTable[Index].key = target

RETURN record

Appropriate operation:.

$$T(n) \approx c$$

TASK 3 - DERIVE THE TIME COMPLEXITY FUNCTION MATHEMATICALLY

3.1 Linear Search

$C(n) = k$ $\therefore$ the target is located at position k

Best case:.

$$C(n) = 1$$

$$T(n) = c$$

$$\therefore T(n) = O(1)$$

worst case.

$$C(n) = n$$

$$T(n) = 4n + c_2$$

$$T(n) = O(n)$$

Average case

$$C(n) = \frac{1+2+3+\dots+n}{n}$$

$$1+2+\dots+n = \frac{n(n+1)}{2}$$

Task 4: Analyse Best, Average and worst case behav

merge sort: Divide and conquer

Best case: Occurs when the array is already sorted or unsorted.

merge sort still divides the array into halves and merges them.

Average case: Occurs when the input elements have a random order

The algorithm continues dividing and merging at every level

worst case: Occurs when the input is an unfavorable order

The recurrence relation is

$$T(n) = 2T(n/2) + \Theta(n).$$

$\therefore$ The best, average and worst cases all have time complexity $\Theta(n \log n)$.

Insertion sort: An array taking one element at a time and inserting into its correct position.

Best case: Occurs when the array is already sorted. Each element requires only one comparison with the previous element.

Average case: Occurs when array elements are in random order.

worst case: Occurs when array is sorted in reverse order.

For the best case

$$T(n) = O(n)$$

For the average and worst cases

$$T(n) = O(n^2)$$

Conclusion: merge sort provides consistent performance for different input arrangements, whereas insertion sort is efficient for small or nearly sorted arrays but become inefficient for large, reverse-sorted datasets.

Tasks: Perform Asymptotic Analysis.

It describes how an algorithm's running time grows as the input size n increases.

It helps compare algorithms independently of specific hardware.

Asymptotic complexity comparison.

Algorithm	Best case	Average case	worst case
merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$

Relative growth of operation counts:

Merge sort: At 1000 records, $n \log_2 n$ is about 9,966. At 1,000,000 records it is about 19,931,569. Its operation count increases roughly 2,000 times.

Task 6: Formulate the Recurrence Relation.

merge sort:

Step 1: original input size

Let n represent the number of elements in an input array.

Step 2: Size of each recursive subproblem

* The array is divided into two equal halves, each of size $n/2$

Step 3: Number of recursive calls.

* Two recursive calls are made, one for each half.

Step 4:

* Merging the two sorted halves requires linear

time

Step 5: Derive the recurrence relation

$$T(n) = 2T(n/2) + O(n)$$

$$\therefore T(n) = O(n \log n) \quad \therefore \text{by using MT Theorem.}$$

Conclusion: merge sort generally offer better scalability and more consistent performance.

Task 7: solve the Recurrence Relation

Binary search;

It is used to search for a student record in a sorted database containing n students records. In each recursive call, the algorithm compares the target value with the middle element and eliminates half to the remaining search space.

Recurrence Relation:

Number of recursive calls: one recursive call on half of the input

Best case:

If the element is found or the search space becomes empty the algorithm terminates in $O(1)$

time

Therefore the recurrence relation is

$$T(n) = T\left(\frac{n}{2}\right) + c$$

Best case: $T(1) = c$

Solving using the iteration method:

$$T(n) = T\left(\frac{n}{2}\right) + c$$

$$T(n) = T\left(\frac{n}{2^k}\right) + kc$$

$$T(n) = T(1) + c \log_2 n$$

$$T(n) = c + c \log_2 n$$

$$\boxed{T(n) = O(\log n)}$$

$$\therefore \frac{n}{2^k} \approx 1$$

$$\therefore n = 2^k$$

$$\therefore k = \log_2 n$$

Task 9 :: Record experimental Performance

Objective :: Evaluate the performance of merge sort and insertion sort in the university student-record system.

Experimental setup ::

1. Application :: university student-record management system.
2. Input data :: student IDs and other details.
3. Algorithm :: merge and insertion sort.
4. Performance measures :: No of comparison and execution time.

Experimental result ::

Input Size	merge sort	Insertion sort
5000	8,700	249,750
10,000	120,000	24,997,500
100,000	1,530,000	2,499,975,000

Conclusion :: merge sort demonstrate better scalability for large student-record datasets, whereas insertion sort is more appropriate for small or nearly sorted inputs.

Task 10: Compare Theory with experimental behaviour

The theoretical analysis predicts how the algorithms grow as the input size increases.

Experimental analysis measures their actual performance when processing student records.

Actual execution time may differ from theoretical predictions for several reasons.

- * Hardware difference

- * Programming language

- * Input order

- * memory overhead

- * measurement errors.

Conclusion: The theoretical analysis and experimental behaviour should show the same overall trend.

Task 11: Recommend and Defend the final design

Merge Sort is the preferred general purpose algorithm, because it provides predictable $O(n \log n)$ performance.

Task 12: *) Insertion sort can still be used for small datasets or nearly sorted records for a production university database. Database indexing and query optimisation should also be considered to improve retrieval.

Task 12: adapt or redesign

Reevaluation of the original Algorithmic design

A. impact analysis :-

The university has changed its requirements to Process 50,000 search requests per hour on a database containing 1,000,000 student records

This could lead to high computation ~~base~~ costs since records are updated only once a day / pre processing techniques such as sorting or indexing.

B. Evaluation of the original design:

The original linear search algorithm has a time complexity of $O(n)$, where n is the number of student records

For, 1,000,000 records a single search may require up to 1,000,000 comparison. For 50,000 searches per hour, the worst case workload.

$$50,000 \times 1,000,000 = 50,000,000,000$$

$\therefore$ Therefore, a linear search is inefficient under the new requirements

* pre processing is acceptable because new records are added only once per day, allowing the system to reuse the prepared data for many searches

C) Adaption or redesign:

Decision: modify the original solution.

The revised strategy:

- * Load the student records into a hash table or
- * when new record added, update the hash table
- * use the prepared structure to handle frequent search

Computational complexity:

- * Linear search: $O(n)$
- * Hash table: $O(1)$
- * Binary search: $O(\log n)$
- * Sorting records: $O(n \log n)$

D Evidence - Based Defence

- * Hash tables can provide average case constant time lookups, making them another efficient option. Since student records are updated only once per day the preprocessing or indexing cost can be shared across many search requests.

Therefore, modifying the original design to use hashing or database indexing is the most suitable decision.